



Arquitectura

Grupo Cordillera

Integrantes :

Juan pablo Rivera

Nayaret Rivas

Profesor :

Israel Alejandro Villagra Riquelme

Tabla de contenido

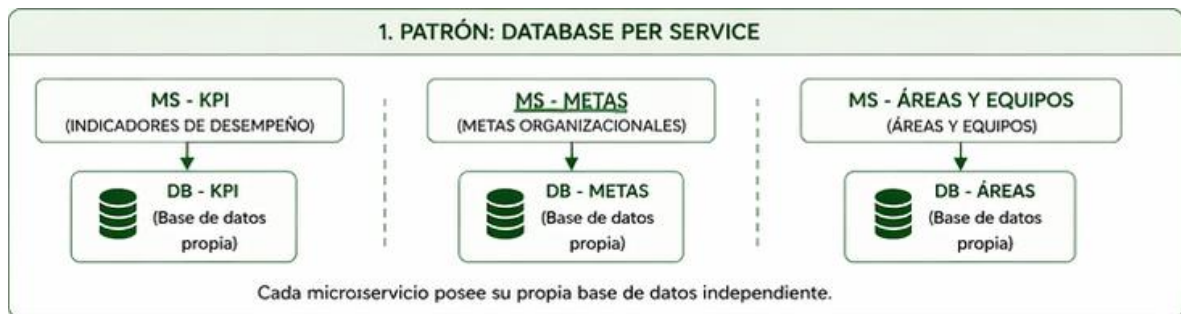
1. Esquema de Selección de Patrones de Arquitectura	3
2. Justificación de herramientas y estrategias de implementación	7
3. Diagrama de la Arquitectura de Microservicios Propuesta	9
4. Consideraciones de Seguridad, Privacidad y Sostenibilidad	16
5. Evaluación General del Diseño Propuesto	17

1. Esquema de Selección de Patrones de Arquitectura

Para transformar la actual fragmentación de información de Grupo Cordillera en una plataforma integrada y escalable, se ha seleccionado una **Arquitectura de Microservicios**. Esta decisión se materializa a través de los siguientes patrones técnicos y de diseño:

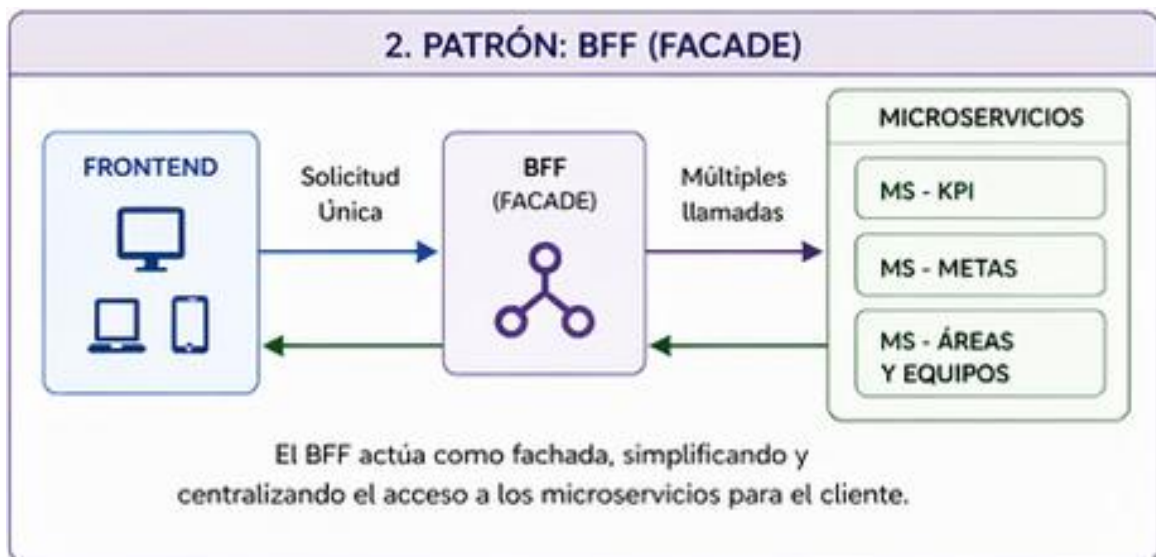
A. Patrón de Microservicios: Database per Service

- **Decisión:** Cada microservicio (KPIs, Metas, Áreas) poseerá su propia instancia de base de datos relacional independiente.
- **Por qué es importante:** En el contexto de retail de Grupo Cordillera, un fallo en el módulo de logística no debe impedir que la gerencia consulte las ventas en tiempo real.
- **Justificación:** Garantiza el **bajo acoplamiento**. Si modificas la estructura de datos de los indicadores de ventas, no afectas al microservicio de gestión de equipos, permitiendo que cada área del negocio evolucione a su propio ritmo sin generar dependencias críticas.



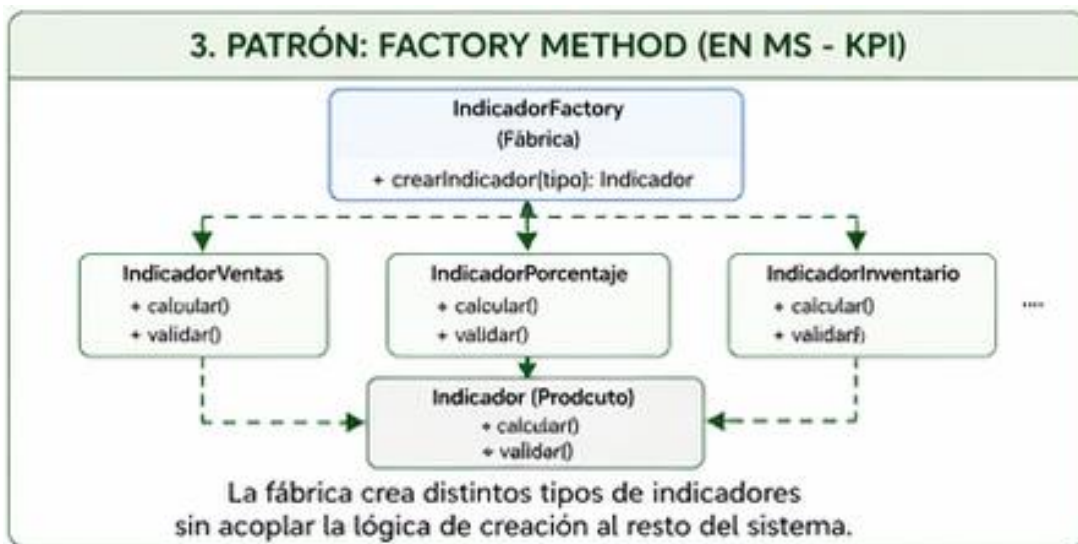
B. Patrón Gateway: Backend for Frontend (BFF)

- **Decisión:** Implementar una capa intermedia (BFF) entre el frontend en React y los microservicios de Spring Boot.
- **Por qué es importante:** Actualmente, la empresa sufre de procesos manuales para consolidar datos de múltiples fuentes. El BFF automatiza esta unión.
- **Justificación:** Actúa como un **orquestador**. En lugar de que el navegador del usuario haga 3 o 4 peticiones distintas para armar un reporte, el BFF realiza esas llamadas internamente en la red, combina los datos (ej: KPI + Meta + Área responsable) y entrega una sola respuesta optimizada. Esto mejora drásticamente el rendimiento percibido por la alta dirección.



C. Patrón de Diseño Creacional: Factory Method

- **Decisión:** Se utilizará para la creación de los distintos tipos de indicadores en el Microservicio de KPIs.
- **Por qué es importante:** Grupo Cordillera maneja indicadores de diversa naturaleza (ventas en CLP, porcentaje de cumplimiento, stock de inventario).
- **Justificación:** Proporciona **flexibilidad y extensibilidad**. Si en el futuro la empresa necesita un nuevo tipo de indicador complejo, el sistema puede integrarlo mediante la "fábrica" sin alterar el código que ya funciona para los indicadores actuales, cumpliendo con el principio de responsabilidad única.



D. Patrón de Resiliencia: Circuit Breaker

- **Decisión:** Implementar interruptores de circuito en la comunicación entre microservicios (especialmente de Metas hacia KPIs).
- **Por qué es importante:** Evita que un fallo en un sistema se propague a toda la plataforma (fallo en cascada).
- **Justificación:** Si el microservicio de KPIs está lento o caído, el patrón **Circuit Breaker** permite que el resto del sistema siga operando (por ejemplo, mostrando datos en caché o un mensaje de error controlado), asegurando que la plataforma de monitoreo sea confiable para la toma de decisiones estratégicas.



2. Justificación de herramientas y estrategias de implementación

Para garantizar una solución robusta y alineada con los estándares de la industria, se han seleccionado las siguientes tecnologías y metodologías de trabajo:

A. Backend: Spring Boot con Java y Maven

- **Decisión:** Utilizar el ecosistema de Spring Boot para el desarrollo de todos los microservicios.
- **Importancia:** La empresa requiere un sistema que consolide datos de ventas, inventarios y logística de manera confiable. Spring Boot es el estándar para aplicaciones empresariales de alto rendimiento.
- **Justificación:**
 - **Productividad:** Gracias a la configuración automática (*Auto-configuration*), podemos levantar microservicios rápidamente, enfocándonos en la lógica de negocio de los KPIs y no en la infraestructura.
 - **Gestión de Dependencias: Maven** permite que todos los integrantes del equipo manejen las mismas versiones de librerías, facilitando la integración de herramientas como Spring Data JPA para la persistencia y Spring Security para la protección de datos.

B. Frontend: React

- **Decisión:** Construir la interfaz de monitoreo inteligente utilizando React.
- **Importancia:** Los ejecutivos de Grupo Cordillera necesitan visualizar información en tiempo real de forma clara y rápida.
- **Justificación:**
 - **Componentización:** React permite crear componentes reutilizables (como velocímetros de KPIs o tablas de cumplimiento), lo que reduce el tiempo de desarrollo y asegura una interfaz consistente en todos los módulos.

- **SPA (Single Page Application):** Ofrece una navegación fluida sin recargas de página, fundamental para un panel de control (Dashboard) donde los datos deben actualizarse constantemente mediante llamadas al BFF.

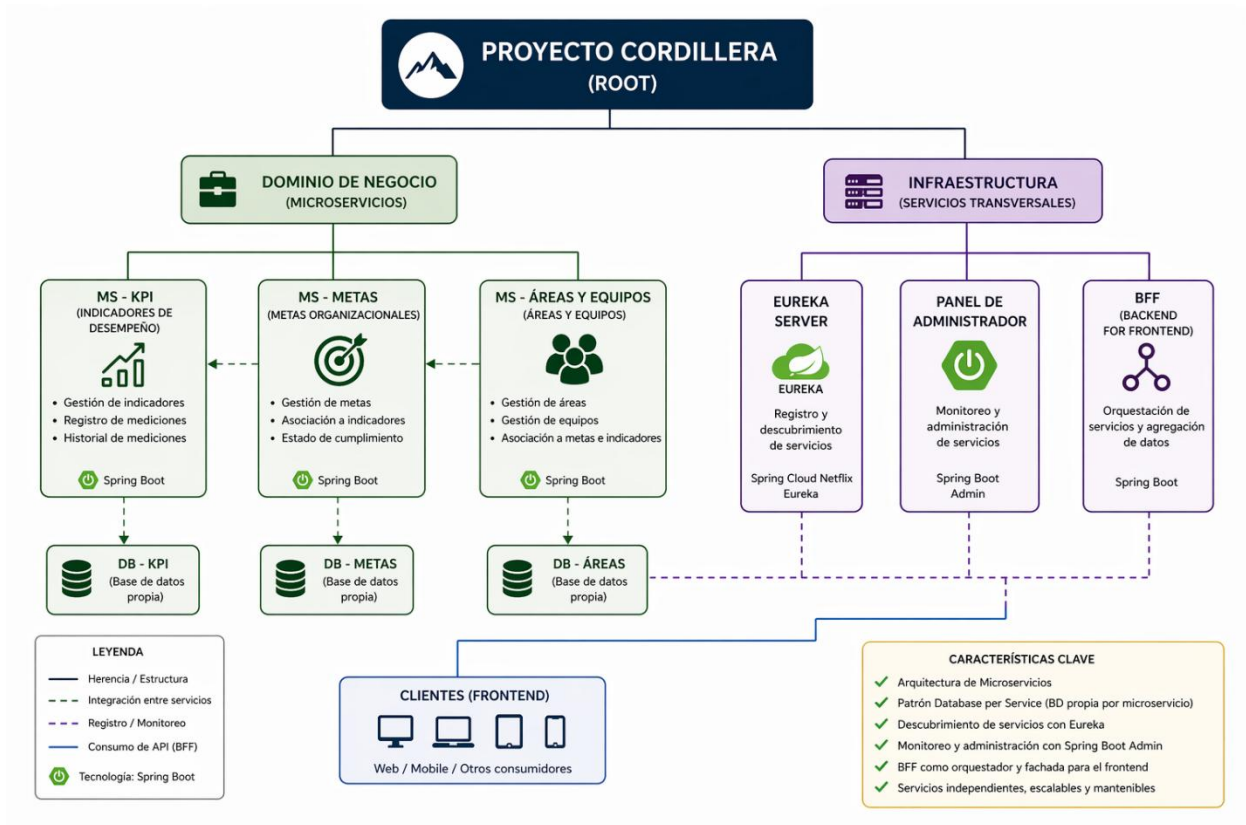
C. Estrategia de Comunicación: RESTful APIs con JSON

- **Decisión:** Estandarizar toda la comunicación entre servicios y el frontend mediante el protocolo HTTP y formato JSON.
- **Importancia:** Actualmente, la información está dispersa en múltiples plataformas (e-commerce, inventarios, finanzas).
- **Justificación:** El uso de JSON como formato de intercambio es ligero y fácil de procesar tanto por los microservicios en Java como por el cliente en React. Esto elimina la fricción en la consolidación de datos y permite una integración más simple con futuros sistemas que la empresa decida añadir.

D. Estrategia de Persistencia: Spring Data JPA (Repository Pattern)

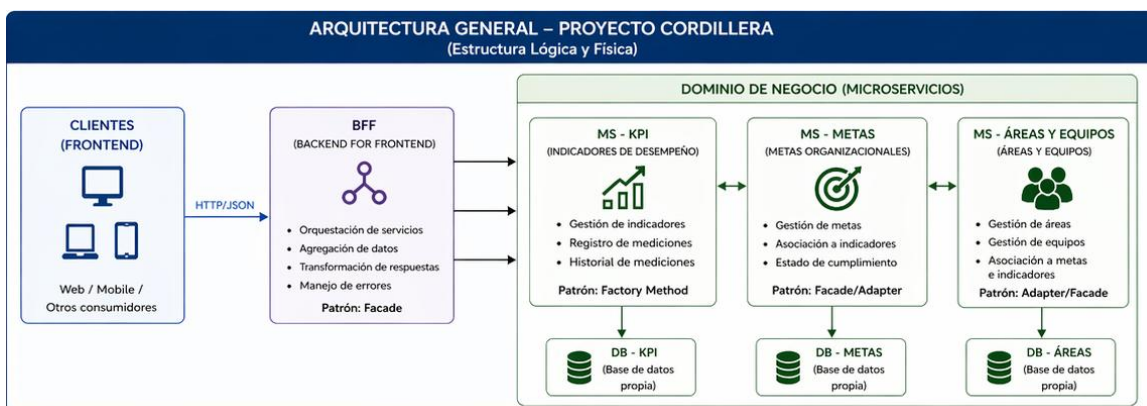
- **Decisión:** Implementar la capa de persistencia utilizando el patrón Repository para abstraer el acceso a las bases de datos relacionales.
- **Importancia:** Grupo Cordillera maneja grandes volúmenes de datos que deben ser validados y almacenados correctamente.
- **Justificación:** Esta estrategia desacopla la lógica de negocio del motor de base de datos específico. Si en el futuro se decide cambiar de base de datos para mejorar la escala, el impacto en el código será mínimo, asegurando la mantenibilidad a largo plazo.

3. Diagrama de la Arquitectura de Microservicios Propuesta



El siguiente diagrama representa la **estructura lógica y física de la solución**, mostrando cómo fluye la información desde la interfaz de usuario hasta la persistencia de datos, dentro de una arquitectura basada en microservicios.

1. Capa de Cliente (Frontend)



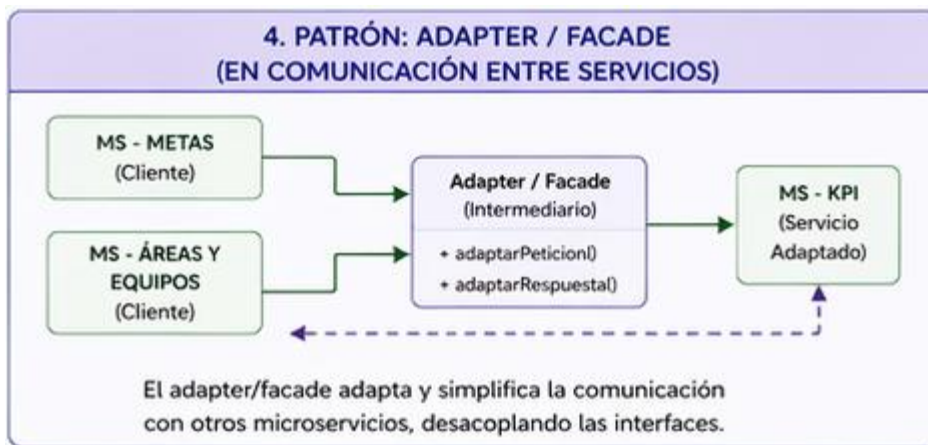
La capa de cliente corresponde a la interfaz de usuario del sistema, la cual puede ser consumida desde navegadores web, dispositivos móviles u otros clientes.

Su principal responsabilidad es:

- Permitir la interacción del usuario con la plataforma.
- Capturar y validar datos antes de enviarlos al backend.
- Consumir los servicios expuestos por el Backend for Frontend (BFF).

El frontend no se comunica directamente con los microservicios, sino que centraliza todas sus solicitudes a través del BFF, simplificando la lógica del cliente.

2. Backend for Frontend (BFF)



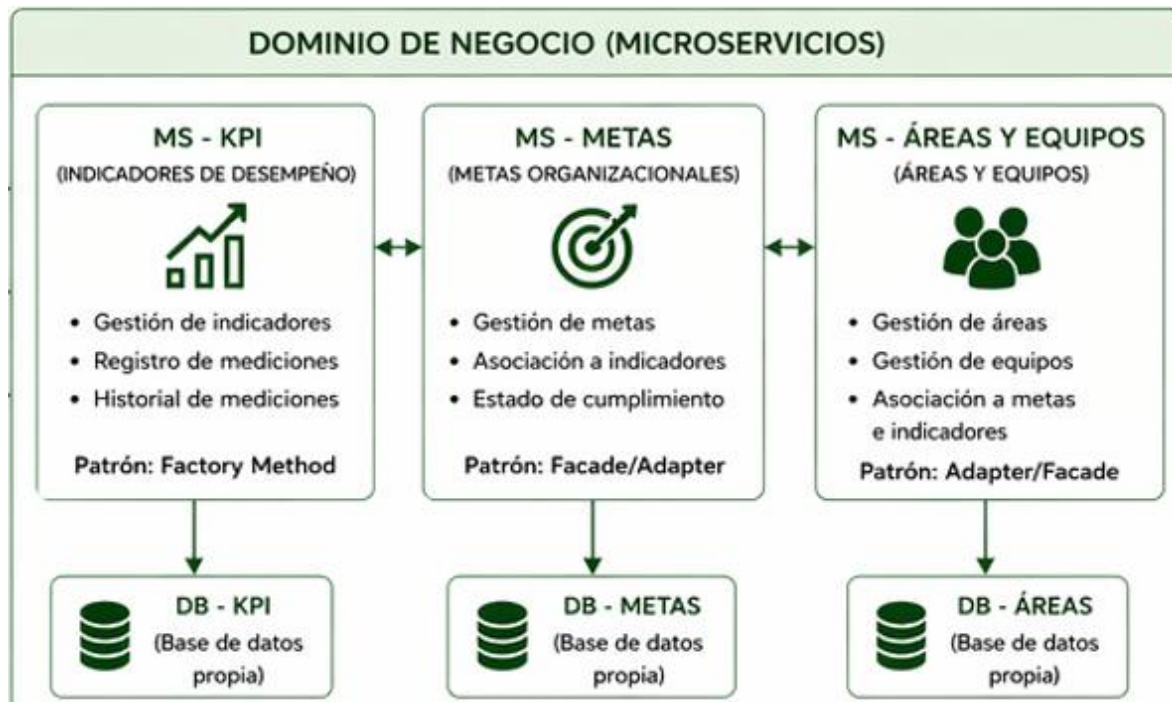
El BFF actúa como una **capa de orquestación** entre el frontend y los microservicios del sistema.

Sus funciones principales son:

- Centralizar las solicitudes provenientes del cliente.
- Orquestar llamadas a múltiples microservicios.
- Agregar y transformar datos para entregar respuestas optimizadas.
- Manejar errores y entregar respuestas consistentes.

Este componente implementa el patrón **Facade**, permitiendo abstraer la complejidad interna del sistema y mejorar el rendimiento al reducir la cantidad de llamadas desde el frontend.

3. Capa de Microservicios (Dominio de Negocio)



La lógica de negocio se encuentra distribuida en tres microservicios independientes, cada uno con responsabilidades claramente definidas:

a. Microservicio de KPIs (Indicadores de Desempeño)

Responsable de la gestión de indicadores organizacionales.

Funciones principales:

- Registro y administración de indicadores.
- Registro de mediciones.
- Consulta de historial de indicadores.

Este servicio implementa un patrón creacional como **Factory Method** para la creación de distintos tipos de indicadores.

b. Microservicio de Metas Organizacionales

Encargado de la gestión de metas asociadas a los indicadores.

Funciones principales:

- Registro y administración de metas.
- Asociación de metas con indicadores.
- Evaluación del cumplimiento de objetivos.

Este microservicio se integra con el servicio de KPIs para validar la existencia de indicadores, utilizando un patrón estructural como **Facade o Adapter**.

c. Microservicio de Áreas y Equipos

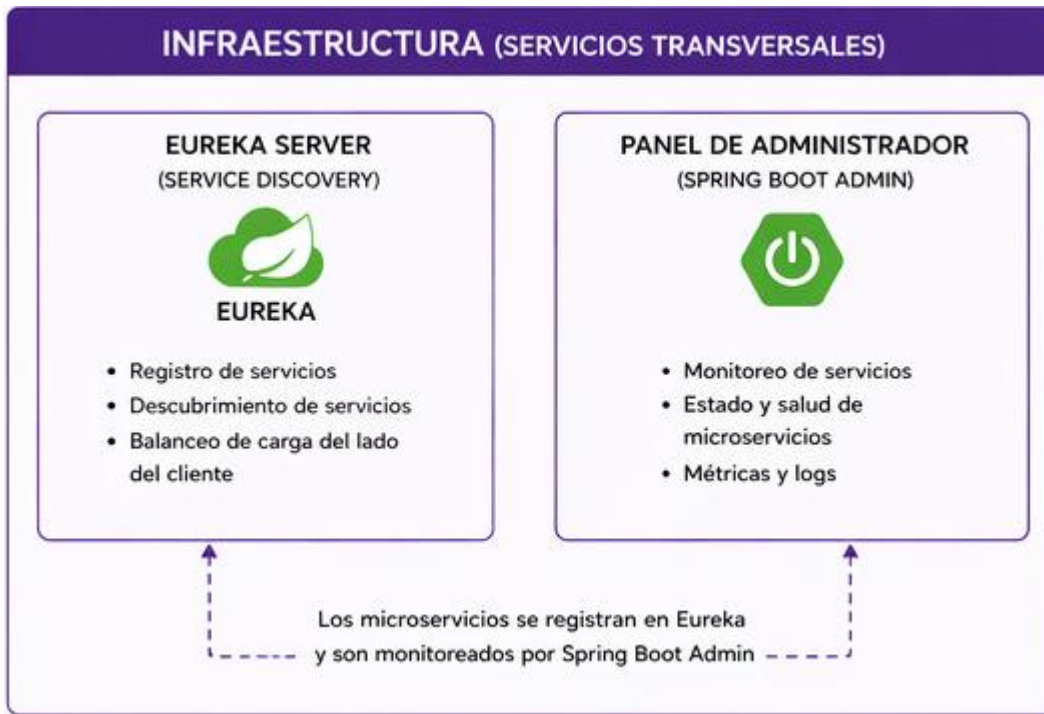
Gestiona la estructura organizacional de la empresa.

Funciones principales:

- Registro de áreas organizacionales.
- Gestión de equipos de trabajo.
- Asociación de equipos con indicadores y metas.

Este servicio se comunica con los microservicios de KPIs y Metas para mantener la consistencia de la información.

4. Capa de Infraestructura



Esta capa agrupa los servicios transversales que soportan el funcionamiento del sistema.

a. Eureka Server

El servidor de descubrimiento permite que los microservicios se registren y localicen dinámicamente dentro de la arquitectura.

Funciones:

- Registro automático de servicios.
- Descubrimiento dinámico de instancias.
- Facilitar la comunicación entre microservicios.

Esto reduce el acoplamiento entre servicios y mejora la escalabilidad.

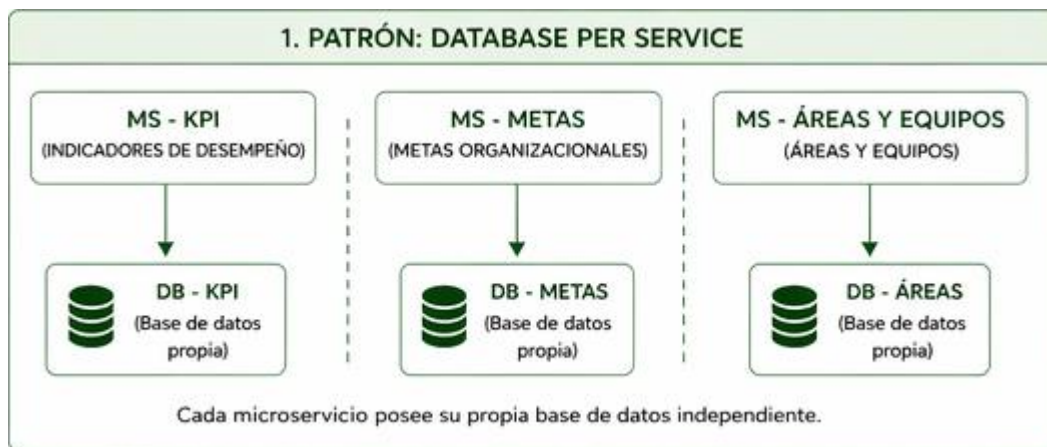
b. Panel de Administrador (Spring Boot Admin)

Permite monitorear y administrar el estado de los microservicios.

Funciones:

- Supervisión del estado de los servicios.
- Visualización de métricas y salud.
- Administración centralizada.

5. Capa de Persistencia (Base de Datos)



Cada microservicio cuenta con su propia base de datos independiente, siguiendo el patrón **Database per Service**.

Características:

- Aislamiento de datos por servicio.
- Bajo acoplamiento entre componentes.
- Mayor escalabilidad y mantenibilidad.

Bases de datos incluidas:

- Base de datos de KPIs.
- Base de datos de Metas.
- Base de datos de Áreas y Equipos.

Flujo de Información en la Arquitectura

El flujo de información dentro del sistema se desarrolla de la siguiente manera:

1. El usuario interactúa con la interfaz frontend.
2. El frontend envía solicitudes al BFF mediante HTTP/JSON.
3. El BFF procesa la solicitud y orquesta llamadas a uno o más microservicios.
4. Los microservicios ejecutan la lógica de negocio correspondiente.
5. Cada microservicio accede a su propia base de datos para persistir o consultar información.
6. Los microservicios retornan los resultados al BFF.
7. El BFF agrega y transforma la información si es necesario.
8. Finalmente, el BFF envía la respuesta al frontend para su visualización



4. Consideraciones de Seguridad, Privacidad y Sostenibilidad

Este apartado detalla cómo la propuesta protege los activos de la organización y asegura la viabilidad a largo plazo de la plataforma.

A. Seguridad (Protección de la Información)

- **Decisión:** Implementación de **JWT (JSON Web Tokens)** y una capa de seguridad en el **BFF**.
- **Importancia:** Grupo Cordillera maneja datos sensibles de ventas, rentabilidad y desempeño de equipos. Un acceso no autorizado podría comprometer la estrategia comercial de la empresa.
- **Justificación:** Al centralizar la autenticación en el **BFF**, los microservicios internos no necesitan gestionar la sesión de usuario individualmente, reduciendo la superficie de ataque. El uso de tokens JWT permite que la comunicación entre el frontend de React y el backend sea cifrada y verificable en cada petición.

B. Privacidad (Gestión de Datos Sensibles)

- **Decisión:** Aplicación de validaciones de datos en capas (Frontend y Backend) y anonimización de identificadores internos.
- **Importancia:** La plataforma registra el desempeño de personas y equipos de trabajo. Es crítico asegurar que solo los jefes de área o ejecutivos autorizados vean información específica de rendimiento individual.
- **Justificación:** El uso del patrón **Repository** permite filtrar los datos antes de que salgan del microservicio, asegurando que el **BFF** solo reciba la información que el usuario tiene permitido ver según su rol, cumpliendo con principios de "privacidad por diseño".

C. Sostenibilidad (Eficiencia de Recursos)

- **Decisión:** Despliegue mediante contenedores y escalabilidad granular de microservicios.
- **Importancia:** Mantener servidores encendidos para procesos manuales o sistemas monolíticos pesados es costoso y energéticamente ineficiente.
- **Justificación:** La arquitectura de microservicios permite escalar solo los módulos con mayor demanda. Por ejemplo, si los lunes hay un pico de tráfico en la **Visualización de Reportes**, se pueden asignar más recursos solo a ese servicio sin necesidad de sobredimensionar toda la infraestructura. Esto reduce el consumo de nube y los costos operativos de la empresa, alineándose con prácticas de tecnología sostenible.

5. Evaluación General del Diseño Propuesto

Este apartado presenta un balance final sobre cómo la arquitectura diseñada responde a los requerimientos del cliente y a las especificaciones técnicas solicitadas.

A. Cumplimiento de Requerimientos del Cliente

- **Eliminación de Procesos Manuales:** Al centralizar la extracción y consolidación de datos en el **BFF** y los microservicios, se eliminan las tareas manuales de transcripción en hojas de cálculo que actualmente generan retrasos y errores.
- **Visibilidad en Tiempo Real:** La integración de un frontend en **React** con servicios backend escalables permite que la alta dirección visualice KPIs críticos (ventas, inventarios, logística) de forma instantánea, cumpliendo con la necesidad de agilidad en la toma de decisiones.
- **Consolidación de Información:** El diseño resuelve la fragmentación actual al unificar datos provenientes de diversas áreas en una única plataforma inteligente de monitoreo.

B. Cumplimiento de Especificaciones Técnicas

- **Modularidad y Separación de Responsabilidades:** El uso de una arquitectura basada en microservicios independientes (KPIs, Metas, Áreas) asegura que cada componente tenga una responsabilidad única, facilitando el mantenimiento y la evolución del sistema.
- **Escalabilidad y Rendimiento:** Al proponer **Spring Boot** con **Maven** y el patrón **Database per Service**, garantizamos que el sistema pueda crecer de forma horizontal según la carga de datos de las distintas sucursales del país.
- **Calidad y Resiliencia:** La incorporación de patrones como **Circuit Breaker** y la estrategia de **Testing** (unitario y análisis de código) asegura que la plataforma sea robusta frente a fallos y cumpla con los estándares de calidad de la industria.

C. Conclusión de Valor Estratégico

- **Justificación de la Inversión:** Aunque la arquitectura de microservicios implica una complejidad inicial mayor que un sistema monolítico, para **Grupo Cordillera** es la decisión correcta. Dada la expansión proyectada de la empresa a nuevos mercados, esta estructura flexible permite añadir nuevas unidades de negocio sin necesidad de reescribir el sistema completo, protegiendo así la inversión tecnológica a largo plazo.
- **Contenerización y Estrategia de Despliegue (Proyección Futura):** Como complemento estratégico, se propone la adopción de contenedorización mediante Docker para el despliegue del sistema. Esta práctica permitiría empaquetar cada microservicio y componente de infraestructura—como Eureka Server y Spring Boot Admin—en unidades independientes, facilitando su portabilidad entre entornos y estandarizando la configuración de ejecución. Adicionalmente, la contenerización favorecería la escalabilidad del sistema, permitiendo replicar servicios según la demanda, así como optimizar los procesos de integración y despliegue continuo. Si bien no forma parte del alcance actual del proyecto, su incorporación futura fortalecería la robustez operativa de la plataforma y alinearía la solución con prácticas modernas de desarrollo y despliegue en arquitecturas de microservicios.